

Paralelni algoritmi - teorija

1. Uvod

Tipovi zavisnosti:

A: $X[i] = 5$ **Prava zavisnost**
... (RAW-Read After Write)
B: $Y[i] = X[i] + 1$ B koristi vrenost koju upisuje A

A: $Y[i] = X[i] + 5$ **Anti-zavisnost**
... (WAR-Write After Read)
B: $X[i] = 42$ B prepisuje vrenost koju koristi A

A: $X[i] = 5$ **Izlazna zavisnost**
... (WAR-Write After Read)
B: $X[i] = 42$ B prepisuje vrenost koju upisuje A

Rešenje za 2 i 3: uvedemo niz X' za proces B

Petlje

Program se sastoji od petlji - spoljašnja i unutrašnja petlja (ključna za performanse programa).

Možemo da paralelizujemo bilo koju od njih.

Razmatranja

1. Telo petlje = poziv 1 metode
2. Koliko iteracija očekujemo
3. Vreme 1 iteracija
4. Zavisnosti unutar i van granica petlje
5. Podaci: odakle + gde idu

Pure (čista) funckija

- Nema bočne efekte
- Rezultat zavisi samo od parametara funkcije

2. Paralelni računari

Murov zakon

Broj tranzistora u integrisanom kolu se udvostručuje na svakih 18 meseci. Problem nastaje 2004. godine => procesori sa više jezgara (*multi kernel processor*)

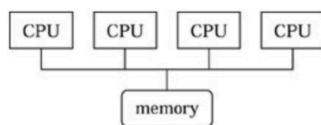
Flinova taksonomija

Klasifikacija računarski sistema na osnovu broja paralelnih instrukcija i kontrolnih tokova.

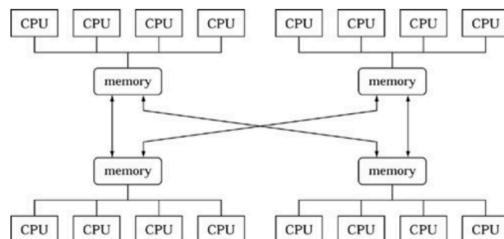
SISD (Single Instruction, Single Data)	MISD (Multiple Instruction, Single Data)
SIMD (Single Instruction, Multiple Data)	MIMD (Multiple Instruction, Multiple Data)

MIMD - podela:

1. Simetrična zaj. memorija

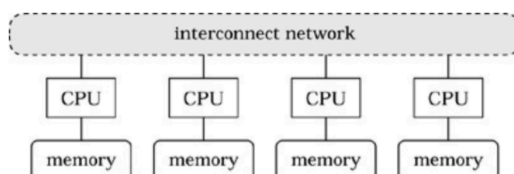


2. Nesimetrična zaj. memorija



3. Distribuirana zaj. memorija

Svaki CPU ima pristup samo lokalnoj memoriji, komunikacija: razmena poruka.



3. Pronalaženje paralelizma

1. Analiza problema

1. Razlaganje problema na manje celine

I Podelom na taskove:

Problem = skup aktivnosti (jasno def. i relativno nezavisne)

Task = poziv metode, telo petlje, obrada 1 elementa niza

Bitno:

Fleksibilnost - skaliranje broja i vremena taska sa podacima i brojem cpu

Efikasnost - da li svaki cpu ima dovoljno posla + da li su svi cpu iskorišćeni

Jednostavna impl. i debug taska

II Podelom podataka:

Korisno kada se problem odnosi na obradu velike strukt. podataka + slične operacije nad delovima (idealno nezavisno).

Primer: nizovi, matrice, tenzori, rek. strukture (stabla, grafovi)

Bitno:

Fleksibilnost - skaliranje broja i veličine delova sa veličinom problema i brojem cpu

Efikasnost - da li svaki cpu ima dovoljno posla + da li su svi cpu iskorišćeni

Jednostavna impl. i debug podele podataka i komunikacije

2. Međuzavisnosti među celinama - koraci

I Grupisanje zadataka

1. 1 grupa = zavisni zadaci

2. 1 grupa = nezavisni zadaci, grupa zavisi od rezultata prethodne grupe

II Utvrđivanje ograničenja pri redosledu zadataka

Najčešće zavisnosti su kroz podatke ili neki spoljašnji servis (npr pisanje u fajl).

III Analiza pristupa i deljenja podataka

1. Lokalni podaci = pristup u okviru 1 zadatka

2. Globalni podaci = pristup iz n zadataka, logički ne pripadaju ni jednom zadatku

* zadatak menja lokalne podatke drugog zadatka

3. Analiza dobijene arhitekture

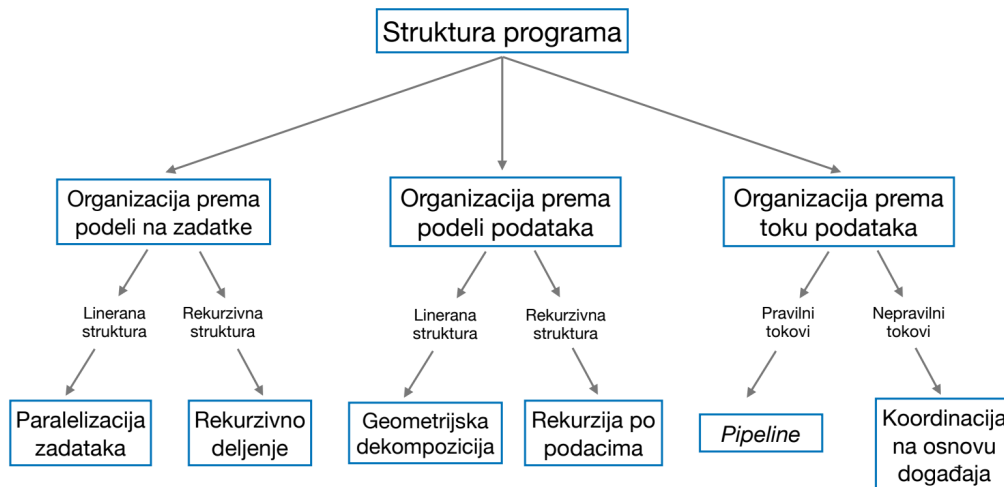
CPU: koliko ih treba, efikasnost, ujednačeno opterećenje

Vreme: korisno izračunavanje vs. sinhronizacija

Deljenje podataka: obrasci za komunikaciju između cpu-jeva

2. Struktura paralelnih programa

Prethodne informacije mapiramo na algoritam i izvršne jedinice.



3. Raspoređivanje poslova na CPU

Statičko

Dinamičko - uniformna raspodela, logika za raspoređivanje se pokreće veći broj puta => **raspoređivanje paketa poslova ***

Statičko sa krađom - jedinica za izvršavanje koja ostane bez posla može uzeti posao iz reda čekanja neke druge jedinice

Vođeno dinamičko - broj poslova se smanjuje svake iteracije *

4. Tehnike paralelizacije

1. Paralelizacija zadataka

Broj i karakteristike zadataka, međuzavisnosti, raspored izvršavanja

2. Rekurzivno deljenje

Zadaci nastaju dinamički u runtime-u, *divide & conquer*

3. Geometrijska dekompozicija

Dele se na segmente (1 segment = 1 cpu), međukomunikacija segmenata sa samim sobom ili susedima, koristi se sliding window.

Veće strukture: niz, matrice

4. Rekurzija po podacima (pointer jumping)

Neefikasnije rešenje nego sekvencijalno ali ga ubrza jer se paralelizuje.

Rekurzivne strukture: stabla, grafovi

5. Pipeline

Algoritam iz n faza istog trajanja (razl. faze za razl. zadatke u paraleli).
Dobro za velike podatke + kad podaci dolaze iz externog izvora.

6. Koordiniranje na osnovu događaja = Aktorski model

Proširenje pipeline modela kada su zadaci neregularni ili nesinhroni.
Receive event - process event - send events

Aktor = računa, čuva podatke/stanje, komunicira sa aktorima

Aktor - primi poruku:

1. Kreira nove aktore
2. Šalje poruku postojećim aktorima
3. Odluči šta će da uradi sa sl. porukom koju primi

4. Niti

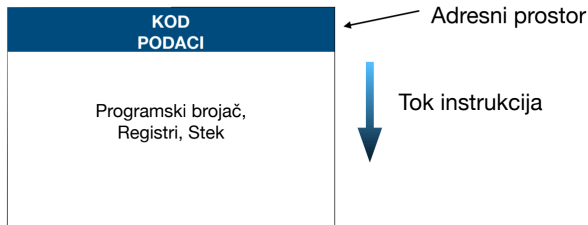
Von Neumanova arhitektura

Registri - fizički, logički

Program ima svoj adresni prostor, potrebna je mapa: virtuelna memorija -> fizička memorija.

Paralelni procesi

PCB - process control block, info za izvršavanje toka instrukcija



1. **Procesi** - svaki proces ima svoj PCB
2. **Niti** - dele adresni prostor, ostalo ima svako svoje

1 jezgro => time slicing (procesor malo izvršava jednu nit pa malo drugu)

N jezgara => time slicing scheduler (procesor može da izvršava više niti)

Hyperthreading

Imamo dodatna logička jezgra (1 fizičko + 1 logičko) koja možemo koristiti kada proces ne zauzima celo fizičko jezgro. Tada na 1 jezgru izvršavamo 2 niti istovremeno, svaka nit ima svoj brojač.

5. Niti u različitim okruženjima - ne za ispit

1. Pthreads - unix sistemi u C-u

```
int pthread_create(pthread_t *tid, pthread_attr_t * attr_t,           // atributi niti
                  void *(* start_routine) (void *),                 // funkcija koju nit izvršava
                  void *arg );                                       // argumenti za funkciju
```

2. Java (ugrađeno, monitor)

Deo standardne biblioteke, nitima upravlja JVM. Dva načina implementacije - *extends Thread* ili *implements Runnable*. Java objekti u sebi imaju lock.

Monitor - keyword *synchronised*:

1. Metode - na nivou instance
2. Statičke metode - na nivou klase
3. Blok koda - samo 1 nit može da izvršava taj blok koda

3. OpenMP - C

Promenljive: *private(A)* - svaki thread ima kopiju A, *shared(B)* - shared

Mutex: # *pragma omp parallel* - paralelni kod

 # *pragma omp critical* - protected

 # *pragma omp atomic* - 1 protected line, operations on scalars

Petlje: # *pragma omp for* - iteracije su raspoređene po nitima

6. Python - niti

Thread klasa

.start - započinjemo nit

.join - blokiramo nit iz koje zovemo dok se nit koju smo pozvali funkcijom ne završi

Race condition

Kada su vrednosti koje dobijemo paralelnim izvršavanjem nedeterminističke, tj zavise od redosleda izvršavanja instrukcija u razl. nitima.

Sinhronizacija putem ključeva

Ključ = lock/mutex

Nit mora da uzme ključ da bi pristupila nekom resursu (čekamo ako je resurs zauzet).

Kritična sekcija = deo između uzimanja (*acquire*) i oslobađanja (*release*) ključa

Deadlock

Nit 1 je uzela ključ 1 i zahteva ključ 2.

Nit 2 je uzela ključ 2 i zahteva ključ 1.

Rešenje 1: izbegavanje držanja 2 ključa u 1 niti

Nije uvek moguće.

Rešenje 2: uzimanje ključeva istim redosledom

Sve niti uvek uzimaju ključeve istim redosledom.

Rekurzivni ključ (RLock)

Dozvoljava da kada smo u kritičnoj sekciji uzmemo isti ključ, samo moramo da releasujemo isti broj puta koji smo uzeli ključ.

Semafori

Generalizacija ključeva, više niti može istovremeno da pristupi 1 resursu - u sebi ima brojač.

.acquire (brojač --)

.release (brojač ++)

Condition (jug-sever prelaz auta)

Mehanizam sinhronizacija vezan za neko logičko stanje u aplikaciji. Jedna nit javlja kad je uslov ispunjen, a drugih n niti čeka. Uslov je uvek vezan za neki lock - ugrađen ili prosleđen u konstrutoru.

.acquire

.wait - oslobađa ključ i blokira se dok neka nit ne notifikuje, tada pokušava da uzme lock

.notify - budi 1 random nit koja čeka

.notifyAll - budi sve niti koje čekaju

—> ovo ne oslobađa ključ, probuđena nit nastavlja kada mi oslobodimo lock

Event

Jedna nit šalje signal a druga nit čeka, imamo interni indikator stanja (flag).

`.set` - flag = 1

`.clear` - flag = 0

`.wait` - blokiramo se dok flag ne postane 1

Queue (FIFO)

`.put` - na kraj, blokirajuće

`.get` - sa početka, blokirajuće

`.put_nowait` - neblokirajuće, baci exception ako je pun queue

`.get_nowait`, neblokirajuće, baci exception ako je prazan queue

`.task_done` - niti kada završe obradu elementa kojeg su uzeli

`.join` - blokira dok se za svaki element iz reda ne zovne `task_done`

Reference counter

Svaki objekat u sebi ima brojač referenci na samog sebe. Ovo onemogućava true threading u pythonu jer bi prečesto morali da lockujemo taj brojač.

GIL - Global Interpreter Lock

Globalni lock koju nit mora da uzme da bi se izvršavala. Ovo garantuje da će samo 1 nit da se izvršava u datom trenutku. Ako ima više niti, menja ih svako malo. Pravo ubrzanje s nitima u pythonu imamo kada čekamo na neke externe dođaje ili resurse.

7. Python - procesi (multiprocessing paket)

Python procesi imaju svoju kopiju interpretera, te i svoj GIL. Tipovi procesa:

1. User - unutar korisničkog adresnog prostora
 - * daemon - pozadinski, za neki servis (root user)
2. Kernel - unutar kernelovog adresnog prostora

Proces se forkuje od roditelja, on je kopija roditelja (ne nasleđuje lockove i podaci o korišćenju resursa su čisti).

Orphan - kad se parent završi pre deteta. Usvaja ga init proces.

Copy on write - dokle god roditelj i dete samo čitaju podatke, koriste istu kopiju podataka. Kad jedno pokuša da promeni podatke, tek tada pravi svoju kopiju zajedničkih podataka. Python uz objekat ima i header gde je info o objektu (broj referenci na objekat). Prilikom pristupa elementu iz drugog procesa, python želi da pristupi headeru i promeni brojač referenci. OS ovo vidi kao izmenu pa pravi kopiju memorijske stranice u kojoj je promenljiva smeštena. Problem sa nizovima i for petljom -> u svakoj iter. napravimo kopiju i prepunimo RAM.

Isti problem **nemamo** sa **numpyom**. Tu je ceo niz jedan Python objekat. Pristup nizu iz drugih procesa izaziva dupliciranje mem. stranice gde je PyArrayObject, ali ne i ostatak niza.

Unix - dete je kopija roditelja

Windows - čist nov proces

Proces klasa

.start, .join

Možemo proslediti demon = True. Ovo nije Unix daemon proces. On se samo gasi kad prekine roditeljski proces umesto da postane siroče.

.exitcode - 0 = proces se završio bez greške

Pozitivan = proces završio uz grešku

Negativan = proces nasilno ugašen sa .terminate ili .kill

Komunikacija između procesa: Pickle modul

Serijalizuje i deserijalizuje python objekte.

Serijalizacija zavisi i od verzije aplikacije => samo u okviru 1 projekta.

U picklovan objekat je moguće ubaciti maliciozan kod => samo za komunikaciju procesa.

`multiprocessing.Pipe`

Sastoji se iz 2 Connection objekta. Po difoltu je jednosmeran, duplex = dvosmeran.

`multiprocessing.Queue (FIFO)`

.put, .get - blokirajuće osim ako se ne prosledi 0 ili False

Deljenje stanja između procesa: Value, Array

Sadrže podatak, info o tipu podatka, ključ za podatak - opcionalan. Trebaju da se koriste lockovi zato što operacije nad podacima nisu atomične => race condition.

multiprocessing.Pool

Pool = skup resurse (procesa). Pool vodi računa o raspoređivanju zadataka na procese - kreiranje, gašenje i broj aktivnih procesa.

.apply - blokirajuća

.apply_async - ne blokira, vraća objekat koji liči na Future. Iz tog objekta posle uzimamo rezultat sa .get i možemo i da čekamo sa .wait => ready, successful

.map - prosleđujemo iterable. Elementi iterabla se serijalizuju i raspoređuju na procese.

Chunksize = po koliko elemenata se šalje 1 procesu.

Usporeenje kada je vreme (de)serijalizacije veliko a funkcija na procesu kratka.

.close - čeka procese da se završe

.terminate - nasilno prekida procese

8. Map Reduce

HDFS - Google file system, otporan na otkaze. Kao naš prvi domaći, fajlove obrađuje u chunkovima u procesima i zapisuje kompresovane chunkove na disk.

Map - map funkcija ne vidi efekte map f-je nad drugim elementima + komutativni pozivi (laka paralelizacija).

Map reduce - za veliku količinu podataka. Pure funkcije, otporan na otkaze, automatska paralelizacija/distr. Rezultat jedne iteracije je ulaz u narednu iteraciju.

result = reduce(funkcija, iterable, starting_result)

Lokalnost - map funkcija se izvršava na mašini sa podacima

Optornost na otkaze - komponenta otkaze -> taskovi se ponavljaju na drugoj mašini, neuspešan task se ponavlja do 4 puta, app može definisati dozvoljen procenat neuspešnih taskova, preskačemo pogrešne key-value parove

Optimizacija (combine faza) - parcijalna redukcija međurezultata pre kopiranja na 2. mašinu

Analiza kompleksnosti - veličina redukcije q, količina replikacije

Primeri: brojanje reči, množenje matrice vektorom, k-means clustering

9. Vektorizacija - jednom u nikad na ispitu

Oblik SIMD izračunavanja - vektorske instrukcije.

1. Upotreba vektorizovanog kompajlera - kad god vidi petlju
2. Intrinsic f-je - kod = specijalizovane vektorizovane f-je
3. Dodavanje vektorskih instruk. u *assembly*

Automatska vektorizacija - ograničenja:

1. Samo za un. petlje
2. Broj iteracija mora biti poznat
3. Samo zavisnosti unutar granica petlje
4. Samo f-je označene za SIMD izvršavanje

Vektorske zavisnosti = zavisnosti preko granica petlje (prave i pretpostavljene)

Funkcije za SIMD izvršavanje - # pragma omp simd

Kompajler generiše 3 vrste ovih instruk.

1. **standardna** - skalarna impl. (skalarni parametri)
2. **vektorska** impl.
3. **vektorska maskirana** impl.

10. Actors, Futures (and Promises) and Ray

1. Aktori

Aktor - osobine:

- Prima poruku i reaguje
- Izolovano un. stanje
- Asinhroni, perzistentni
- 1 trenutak = 1 poruka se obrađuje

Aktor obavlja: računanje, čuva un. stanje, komunicira sa drugim aktorima

Aktor primi poruku:

- Kreira nove aktore
- Pošalje poruke postojećim aktorima
- Odluči šta će uraditi sa sl. porukom koju primi

Komunikacija:

- At most once delivery (nema automatski resend)
- Best effort delivery
- Nema time limit
- Nema garancije za redosled poruka

Aktor može imati više adresa.

Supervizija

Greška -> aktor šalje poruku svom supervizoru: ponovo obradi, preskoči, ugasi i reset aktora, super()

Ask and tell: komunikacija sa sp. svetom

Ask - send mssg to actor & wait

Tell - send mssg to actor & not wait

Create actor - creates master actor

Thespian: python framework za rad sa aktorima

2. Futures and Promises

Future = mesto za vrednost koja nije vraćena (možemo da ulančavamo)

Promise = funkcija koje će biti izvršena i upisuje rezultat u future

Sa future objektom radimo sve dok vrednost ne postane neophodna. Greška se vidi tek pri pristupu vrednosti. **Korisno** za request-response procedure, UI/UX, dugačka izračunavanja, upiti ka bazi, čitanje podataka sa socketa.

3. Ray

Okruženje za razvoj paralelnog i distr. softvera - mikroservisi i aktori, otpornost na otkazivanje, big data,...

Osnovni koncepti

Funkcija = task -> vraća Future koji može biti parametar f-ja

Klasa = aktori

Formira se graf izračunavanja (zavisnosti) - kako podaci teku. Izvršavanje je tek na kraju programa pri pozivu ray.get().

11. Dataflow - ne za ispit

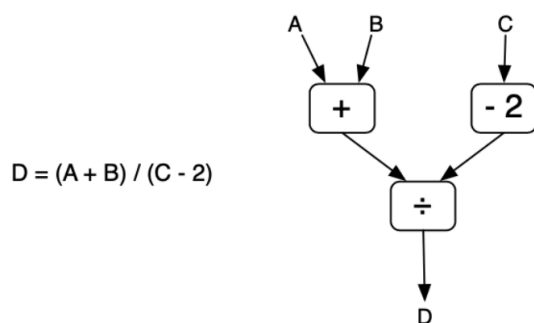
Računarstvo bazirano na *toku podataka* umesto na vremenskoj sekvenci operacija. Operacija se izvršava kad su svi operandi spremni, ne postoji brojač.

Wavescalar - dataflow računar, nije komercijalan, decentr. izvršavanje

Program = graf kontrole toka

Čvorovi = instrukcije

Ivice = statičke zavisnosti između instrukcija, 'tok' rezultata



Procesori:

1. **Statički** - 1 ivica = 1 vrednost

2. **Dinamički** - 1 ivica = više vrednosti (više instanci čvora se izvršava u 1 trenutku)

Problemi:

1. **Petlje** -> tok podelimo na talase (logičke celine) = 1 iter. petlje

2. **Prisup memoriji** -> pristup između ili unutar talasa

12. CUDA

1. Uvod

Framework za pisanje koda koji se izvršava na GPU.

CPU ima veliku keš memoriju -> optimizovan za sekv. izvršavanje.

GPU ima veliki broj jezgara -> optimizovan za paralelne tokove instrukcija.

Arhitektura:

1. **SM** (streaming multiprocessor)
2. **SP** (streaming processor) = cuda jezgro, deli kontrolnu logiku i instruk. keš
3. **GDDR** (globalna mem.)

Struktura cuda programa

Host - sekvencijalni deo na CPU + **device** - paralelni deo na GPU

Kernel funkcija u GPU se izvršava na 1 CUDA niti. Podela niti:

1. **Grid** - skup svih niti
2. **Blok** - do 2048 niti
3. **Warp (snop)** - 32 niti, osnovna jedinica izvršavanja i dele istu memoriju, sve niti rade na 1 tasku (za razliku od cpu gde svaka nit može imati svoj task)

Pri izvršavanju, kernel funkcija ima grid. Svaki blok zauzima jedan SM. Svaki warp zauzima jedan SP (cuda jezgro).

2. Memorija

Upravljanje memorijom (host vodi računa o memoriji):

1. **Globalna** (r-w) - ukoliko nit menja podatak u globalnoj memoriji, nit iz istog kernela ne treba da čita tu vrednost
2. **Konstantna** (r) - svaki blok (SM) ima svoju malu konstantnu memoriju
3. **Deljena memorija** - na nivou bloka (SM), koristimo `_syncthreads_` da bi ručno sinhronizovali snopove u bloku

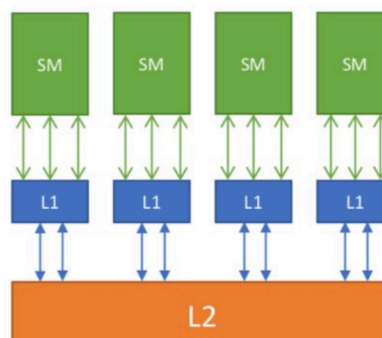
GPU keš ima L1 i L2 nivo. Svaki SM ima svoj L1 keš, dok je L2 zajednički za ceo GPU.

Jedna keš linija je 128 bajtova. U odnosu na CPU ima:

1. Manji keš
2. Veće kašnjenje
3. Veći protok

Pristup kešu:

1. Gusti - vraća celu keš liniju
2. Retki - vraća samo taj podatak



Kod:

- __device__ - globalne promenljive vidljive ostalim kernelima
- __constant__ - skladišti se u veoma brzu konstantnu memoriju (*read only*)
- __shared__ - deljena memorija u SM, preporučljiva statička alokacija

Registri - skladište promenljive deklarisanе unutar kernela. Svaki SM ima 32-bitne registre => postoji ograničenje broja registrara po bloku i nitima. Kada pokušavamo da koristimo više registra, dolazi do **prelivanja** u globalnu memoriju.

3. Tok izvršavanja i sinhronizacija

Komande:

- threadIdx.(x / y/ z) = koordinata niti u bloku
- blockDim.(x / y/ z) = dimenzija bloka, koliko niti ima u toj dimenziji
- blockIdx.(x / y/ z) = koordinate bloka u grid sistemu

Primeri:

gridDim.X = 4, gridDim.Y = 3 i gridDim.Z = 2 => $4 * 3 * 2 = 24$ bloka u gridu
 blockDim.X = 16, blockDim.Y = 8 i blockDim.Z = 4 => $16 * 8 * 4 = 512$ niti u bloku

Uslovne instrukcije -> koristimo mehanizme **divergencije niti**

Kreiramo nit za svaku opciju. One čiji uslov nije ispunjen ne rade ništa, maskirane su.
 Divergencija: if-else, case, for

Sinhronizacija niti:

- syncthreads - na nivou bloka
- sinhronizacija niti na nivou grida (samo iz hosta), čekamo da se završi kernel

Atomične operacije - jedinične, garantuju konzistentnost

Memorijske operacije u cudi su **neblokirajuće**. Ako želimo konzistentnost podataka, dodamo *threadfence*, blokiramo dok se sve mem. operacije do sada ne završe.

4. Redukcija - komutativno i asocijativno

Prvo radimo **lokalnu** redukciju unutar bloka. Rezultat svakog bloka se upisuje na lokaciju u globalnom nizu. Parcijalni rezultati se obrađuju novim pozivom parcijalne redukcije ili na host uređaju - **globalna** redukcija.

Unutar snopa možemo direktno razmenjivati podatke. Koristimo *_shfl* varijante.

Mogući problemi:

- Divergencija niti, polovina niti npr. ne radi ništa
- Konflikti na nivou memorijskih banki, cilj nam je da raspored bude gust

13. Zadaci

1. Zavisnosti van granica petlje

Tražimo prave zavisnosti i antizavisnosti.

2. Računanje ubrzanja koda

γ - procenat sekv. dela

P - broj procesora

Amdalov zakon

$$S = \frac{1}{\gamma + \frac{1-\gamma}{P}}$$

Gustafsonov zakon

$$S = P + (1 - P) * \gamma$$

- veličina problema se usložnjava
kad se dodaju dodatni procesori

Efikasnost

$$E = \frac{S}{P}$$

3. Problemi paralelizacije, sinhronizacije (3. i 4. vežbe)

Readers - Writers

Dva ključa za sinhronizaciju pristupa deljenom objektu - ekskluzivni (za vrstu usera) i lock za broj readera. Ovako, pisac ima ekskluzivni pristup za pisanje. Više čitača mogu zajedno čitati - dokle god postoje čitači (održavamo brojač), čitači drže ekskluzivni lock.

Producer - Consumer

Tri semafora:

Empty - omogućava proizvođaču da pristupi praznim slotovima u kolekciji

Full - omogućava potrošaču pristup punim slotovima

Mutex - za pristup deljenoj kolekciji radi izbegavanja race conditiona

full = threading.Semaphore(0) na početku da bi forsiralo potrošača da sačeka dok proizvođač prvo spremi proizvod.

Atomic broadcast

Semafori:

Empty - na početku 1 jer je kolekcija proizvoda prazna -> prvo proizvođač

Full - po 1 semafor na svakom proizvodu u kolekciji (start - 0)

Globalna promenljiva **read** - koliko proizvoda smo uzeli (lock za to)

Želimo da prvo proizvođač popuni sve proizvode (svi **full.release**). Potrošači uzmu lock za svoj proizvod povećaju **read**. Kada uzmemo sve proizvode, **empty.release** i **read = 0**.

Dining philosophers

Ključno je da filozof uzme obe viljuške pre početka jela. Deadlock se izbegava tako što svaki filozof (nit) uzima lockove u istom redosledu (leva pa desna viljuška).

Dining savages

Semafori:

Empty / full - signalizira da je lonac prazan / pun

Mutex - omogućava da 1 divljak u trenutku može da priđe loncu

Na početku je lonac pun (full semafor = 1). Divljak uzme mutex pokušava da jede.

Lonac je prazan -> empty.release i full.acquire. Probudi kuvara, ovaj napuni lonac (empty i full obrnuto). Pušta mutex i dolazi naredni.

The bear, the honeypot and the bees

Semafori:

Empty / full - signalizira da je košnica prazna / puna

Mutex - omogućava da 1 pčela u trenutku priđe košnici

Na početku je košnica prazna (empty semafor = 1). Pčele donose med. Ako je puna košnica, onda empty.acquire i full.release. Tad se medved budi, pojede sve i obrne semafore. Pčela pusti lock i dolazi naredna.

Build H2O

Čuvamo globalne promenljive koliko imamo hydro i oxy. Imamo mutex da bi samo 1 atom mogao nešto da radi. Svaki atom dodaje sebe i proverava da li je ispunjen uslov za H2O. Ako je ispunjen, onda releasuje semaphore 2 puta za H i jednom za O. Pušta mutex da može neko drugi da doda sebe. A u nastavku atom 'javlja' da je uradio bond kad acquireuje svoj semafor.

Child care problem

Od vrednosti čuvamo broj dece, dadilja i wanting_to_leave - treba nam mutex za ove vrednosti. Imamo i 1 semafor can_leave za broj dadilja koje mogu da odu. Imamo 4 akcije: **bring_kids**: ako ima dovoljno dadilja, dodajemo decu i započinjemo nit za odvođenje dece. Ako nema, ispisujemo da ne može.

take_kids: Oduzmemo decu i dokle god možemo, dopuštamo dadiljama da odu (wanting_to_leave-- i nanny-- i can_leave.release) *.

come_to_work: Povećamo broj dadilja i *.

go_home: Ako možemo, odmah odemo kući (nanny--), u suprotnom povećamo wanting_to_leave. Kada acquireujemo can_leave, tada smo otišli kući.

One-lane bridge problem (north and south)

Može da čeka najviše n vozila iz jednog smera. Imamo condition i za svaku stranu brojače čeka, prelazi i prešlo. Sa jedne strane:

Dokle sa suprotne strane prelaze ili je max naših prešlo, povećavamo naše 'čeka' i waitujemo. Dobili condition: čeka-- prelazi++ i resetujemo suprotno[prešlo] na 0. Ako neko sa druge strane čeka, omogućavamo mu da pređe most sa mi[prešlo]++. Puštamo condition. Potom mi prelazimo.

Nakon što smo prešli, uzimamo condition i mi[prelazi]--. Ako niko naš više ne prelazi, javljamo svima da sve pođe ispočetka.

Cigarette smokers problem

Imamo 2 conditiona - ready, empty.

Agent uzima ready, postavlja na sto i javlja svima sa ready.notifyAll i ready.release. Potom čeka da pušači završe sa empty.wait i nastavlja u naredni krug.

Pušač uzima ready. Dok mu ne odgovaraju stvari čeka sa ready.wait. Javlja da je završio sa empty.notifyAll, i na kraju pušta ready lock.

4. Da li je moguća vektorizacija (7. vežbe)

Zadaci 5, 6 i 7 - da li je moguće vektorizovati sledeći kod?

5 - $A[i] = A[i] + A[i+1]$

može, za trenutni element samo čitamo vrednost narednog, koristimo njegovu nepromenjenu vrednost za izračunavanje

6 - $A[i] = A[i] + A[i-1]$

ne može, za trenutni element nam treba već izračunata vrednost prethodnog

7 - if ($i \% 2 == 0$)

$A[i] = A[i] + B[i]$

Automatska vektorizacija ne može da se snađe sa uslovima. Ovde bi mogli da stavimo da i preskače svaki drugi ($i += 2$) ili da sredimo nizove na neki način.

Ako želimo da radi automatska vektorizacija, treba da uključimo *-Ofast* flag prilikom pokretanja programa.